

Task No.	Task 2(B)
Title	Interactive To-Do List using DOM and Event Listeners
Course Outcome	CO1

1. Aim

To develop an interactive To-Do List web application using JavaScript, DOM manipulation, and event listeners, allowing tasks to be added, marked as completed, and deleted dynamically.

2. Objective

To understand and apply DOM element selection, dynamic creation and removal of elements, event listener handling for click and keypress events, and dynamic class-based styling to build a fully interactive webpage.

3. Theory

The Document Object Model (DOM) lets JavaScript read and change a webpage's structure while it is running. `document.getElementById()` selects existing elements such as the input box and the task list, while `document.createElement()` builds new elements — a list item, a text span, and Complete/Delete buttons — entirely at runtime.

Event listeners, added using `addEventListener()`, let the page respond to user actions: a click on “Add Task” reads and validates the input, creates a new `<li>` with its own Complete and Delete buttons, and appends it to the list using `appendChild()`; a keypress listener also allows tasks to be added by pressing Enter.

Each task's own Complete button toggles a “completed” CSS class using `classList.toggle()`, which strikes through the text and changes the left border colour, while each Delete button calls `remove()` on its own list item to take it out of the DOM. A helper function checks the list's length after every change to show or hide an “empty list” message.

4. Algorithm

1. Start.
2. Create a valid HTML5 document and set the page `<title>` to “To-Do List”.
3. Add a text input for entering a task and an “Add Task” button, followed by an empty task list and an “empty list” message.
4. Select the input, button, task list, and empty-message elements using `getElementById()`.
5. Define a helper function that shows the empty message only when the task list has no items.
6. Add a click event listener to the “Add Task” button.
7. Inside the handler, read and trim the input value; if it is empty, alert the user and stop.
8. Dynamically create a list item with a text span and Complete/Delete buttons using `createElement()`.
9. Add a click listener on the Complete button that toggles a “completed” class on its list item.
10. Add a click listener on the Delete button that removes its own list item from the DOM.
11. Append the new list item to the task list, clear the input box, and update the empty message.
12. Add a keypress listener on the input so pressing Enter also adds the task.
13. Save the file, open it in a browser, and test adding, completing, and deleting tasks.
14. Stop.

5. Program (Source Code) — task2(B).html

task2(B).html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>To-Do List</title>
<style>
  body {
    font-family: 'Segoe UI', Arial, sans-serif;
    background: #f0f4f8;
    display: flex;
    flex-direction: column;
    align-items: center;
    padding: 40px;
  }
  .container {
    background: #fff;
    padding: 25px 30px;
    border-radius: 10px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    width: 380px;
  }
  h2 {
    text-align: center;
    color: #2c3e50;
    margin-top: 0;
  }
  .input-row {
    display: flex;
    gap: 8px;
    margin-bottom: 15px;
  }
  .input-row input {
    flex: 1;
    padding: 8px;
    border: 1px solid #ccc;
    border-radius: 5px;
  }
  .input-row button {
    padding: 8px 14px;
    background: #2c7be5;
    color: #fff;
    border: none;
    border-radius: 5px;
    cursor: pointer;
  }
```

```
}  
.input-row button:hover {  
  background: #1a5cbf;  
}  
#taskList {  
  list-style: none;  
  padding: 0;  
  margin: 0;  
}  
#taskList li {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  background: #f8f9fb;  
  padding: 8px 12px;  
  border-radius: 6px;  
  margin-bottom: 8px;  
  border-left: 5px solid #2c7be5;  
}  
#taskList li.completed span {  
  text-decoration: line-through;  
  color: #888;  
}  
#taskList li.completed {  
  border-left-color: #2ecc71;  
}  
.task-text {  
  flex: 1;  
  word-break: break-word;  
}  
.btn-group {  
  display: flex;  
  gap: 6px;  
  margin-left: 8px;  
}  
.complete-btn, .delete-btn {  
  border: none;  
  padding: 5px 10px;  
  border-radius: 4px;  
  color: #fff;  
  cursor: pointer;  
  font-size: 13px;  
}  
.complete-btn {  
  background: #2ecc71;
```

```
}
.complete-btn:hover {
  background: #27ae60;
}
.delete-btn {
  background: #e74c3c;
}
.delete-btn:hover {
  background: #c0392b;
}
#emptyMsg {
  text-align: center;
  color: #888;
  margin-top: 10px;
}
</style>
</head>
<body>

<div class="container">
  <h2>My To-Do List</h2>
  <div class="input-row">
    <input type="text" id="taskInput" placeholder="Enter a task...">
    <button id="addBtn">Add Task</button>
  </div>
  <ul id="taskList"></ul>
  <p id="emptyMsg">No tasks available.</p>
</div>

<script>
  const taskInput = document.getElementById('taskInput');
  const addBtn = document.getElementById('addBtn');
  const taskList = document.getElementById('taskList');
  const emptyMsg = document.getElementById('emptyMsg');

  function updateEmptyMessage() {
    emptyMsg.style.display = taskList.children.length === 0 ? 'block' :
    'none';
  }

  addBtn.addEventListener('click', function () {
    const taskText = taskInput.value.trim();

    if (taskText === '') {
      alert('Please enter a task.');
```

```
}

const li = document.createElement('li');

const span = document.createElement('span');
span.classList.add('task-text');
span.textContent = taskText;

const btnGroup = document.createElement('div');
btnGroup.classList.add('btn-group');

const completeBtn = document.createElement('button');
completeBtn.textContent = 'Complete';
completeBtn.classList.add('complete-btn');

const deleteBtn = document.createElement('button');
deleteBtn.textContent = 'Delete';
deleteBtn.classList.add('delete-btn');

completeBtn.addEventListener('click', function () {
  li.classList.toggle('completed');
});

deleteBtn.addEventListener('click', function () {
  li.remove();
  updateEmptyMessage();
});

btnGroup.appendChild(completeBtn);
btnGroup.appendChild(deleteBtn);
li.appendChild(span);
li.appendChild(btnGroup);
taskList.appendChild(li);

taskInput.value = '';
taskInput.focus();
updateEmptyMessage();
});

taskInput.addEventListener('keypress', function (e) {
  if (e.key === 'Enter') {
    addBtn.click();
  }
});
```

```
updateEmptyMessage();  
</script>  
  
</body>  
</html>
```

6. Expected Output

The screenshot below shows task2(B).html rendered in a Chromium browser after adding two tasks — “I am dancing” and “Eating” — and clicking Complete on “Eating”, showing its strike-through text and green border, demonstrating that tasks can be added and marked complete dynamically through the DOM.

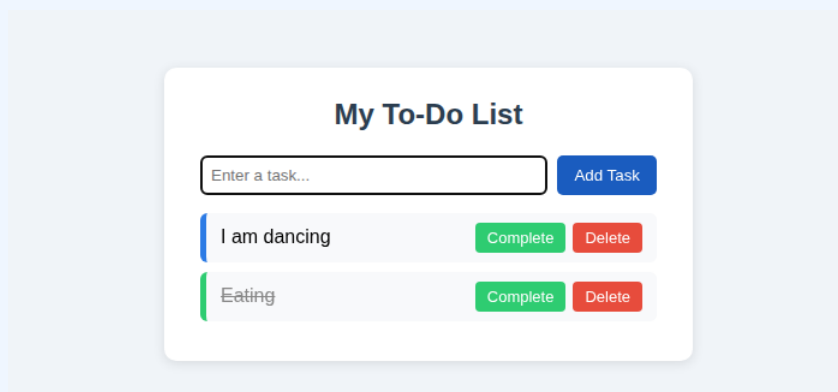


Figure 1: To-Do List — two tasks added, “Eating” marked as completed.

7. Result

The Interactive To-Do List was successfully developed and executed using JavaScript, DOM manipulation, and event listeners. New tasks were dynamically created and added to the list, the Complete button correctly toggled a completed style on each task, and the Delete button correctly removed tasks from the page, with the empty-list message appearing and disappearing as expected — fulfilling Course Outcome CO1.

Student Signature

Faculty / Lab In-charge Signature